Ders 10 Genişletilmiş Notu: Web Taraması: Tarayıcı Türleri, Uygulama Sorunları ve Tercihli Tarama

Ders 10 Genişletilmiş Notu: Web Taraması: Tarayıcı Türleri, Uygulama Sorunları ve Tercihli Tarama

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Kapsamı
- 2. Tarayıcılara Neden İhtiyaç Var?
- 3. Durdurma Koşulları
- 4. Tohum URL Seçimi ve Taramanın Ölçeği
- 5. robots.txt Dosyası
- 6. Tarayıcılar Nerede Kullanılır?
- 7. Tarayıcı Sınıflandırması
- 8. Temel Sıralı Tarayıcı
- 9. Statik Dolaşma Stratejileri: BFS ve DFS
- 10. Örnek Bir Tarayıcı Uygulaması
- 11. Uygulama Sorunları
- 12. Sayfa Deposu
- 13. Eş Zamanlı Tarayıcılar
- 14. Evrensel Tarayıcılar
- 15. Tercihli Tarayıcılar
- 16. Tercihli Tarayıcıları Karşılaştırmak: Tarama Boyutu ve Duyarlılık
- 17. Topical Tarayıcı Örneği
- 18. Açık Kaynak Tarayıcılar

1. Dersin Kapsamı

Bu derste 8. bölüm (Chapter 8), yani web taraması (web crawling) işlendi. Hoca dersin iki eksende ilerleyeceğini söyledi. Birincisi, bir web tarayıcısı (web crawler) yazarken çözmemiz gereken uygulama (implementasyon) problemleridir. İkincisi, ölçektir: Bir tarayıcı küçük ölçekli ve belli bir odak doğrultusunda yazılabileceği gibi, çok daha büyük ölçekli de yazılabilir. Büyük ölçekli tarayıcıların çözmesi gereken problemler ayrıca konuşuldu.

Bu çerçevede üç tarayıcı ailesi ele alındı:

- **Evrensel tarayıcılar (universal crawlers):** Bugünkü arama motorlarını destekleyen tarayıcılar.
- **Odaklı tarayıcılar (focused crawlers):** Belirli bir alana yönelmiş tarayıcılar.
- **Gerçek zamanlı çalışan tarayıcılar:** Arama kelimelerini girdiğiniz anda çalışmaya başlayan tarayıcılar.

2. Tarayıcılara Neden İhtiyaç Var?

2.1 Arama Ters Dizin Üzerinde Yapılır

Hoca soruyu daha önce konuşulan bir noktayla cevapladı: Bir arama motorunda arama yaptığımızda bu arama, elde edilmiş ham sayfalar üzerinde gerçekleşmez. Arama bir ters dizin (inverted index) tablosu üzerinde gerçekleşir. Bu tabloyu oluşturabilmek için önce farklı sunucularda duran HTML sayfalarını belirli bir yönteme dayanarak toplamak ve merkezi bir ortamda saklamak gerekir.

> **Ek bilgi (derste anlatılmadı):** Ters dizin, her kelimeden o kelimenin geçtiği belgelere giden bir eşlemedir. Normal bir belgede belge kelimeleri içerir; ters dizinde ise ilişki ters yönde kurulur. Bu sayede sorgudaki kelimenin geçtiği belgeler bütün sayfalar taranmadan bulunabilir.

İşte tarayıcının tanımı buradan çıkar. Dağıtık ortamda, yani web'de farklı sunucular üzerinde duran HTML sayfalarını belirli bir yönteme dayalı olarak bulup indiren ve merkezi bir saklama ortamında saklayan uygulamalara tarayıcı (crawler) denir. Ters dizini oluşturacağımız sayfalara bu uygulamalar sayesinde ulaşırız.

Bu uygulamaların başka adları da vardır. Crawler, spider, robot ve web agent aynı şeyi ifade eder. Örneğin Google'ın tarayıcısına Googlebot, MSN'in tarayıcısına MSNbot adı verilmiştir.

2.2 Tarayıcının Temel Döngüsü

Bir tarayıcı girdi olarak bir URL kümesi alır. Bunlara tohum URL (seed URL) veya çekirdek URL denir. Tarama şöyle ilerler:

1. Tarayıcı verilen bağlantılardan başlar. Bu bağlantıların bulunduğu HTTP sunucularına HTTP isteği (HTTP request) gönderir.
2. Sunuculardan bu URL'lerin işaret ettiği HTML sayfalarını alır.
3. Sayfaları ayrıştırır (parse) ve içlerinden yeni URL'ler çıkarır.
4. Yeni URL'lere yine HTTP isteği gönderir, gelen sayfalardan yine yeni URL'ler çıkarır.

Yani başlangıçta bir URL kümemiz vardı. Bunların işaret ettiği sayfaları indirdik, içlerinden yeni URL'ler çıkardık ve bu URL'lerle tekrar web'e gittik. Bu işlem sürekli tekrarlanır. Doğal olarak tarayıcıya ne zaman duracağını da söylememiz gerekir.

3. Durdurma Koşulları

Tarayıcıya farklı durdurma koşulları (stopping conditions) verilebilir:

- **Süre:** "Tohum URL'lerden başlayarak 3 saat boyunca ulaşabildiğin bütün web içeriğini tara" diyebiliriz.
- **Belge sayısı:** Toplamda indirilecek HTML belgesi sayısı sınır olarak verilebilir.
- **Veri miktarı:** Saklama ortamının bir üst sınırı varsa "toplamda şu kadar terabaytlık veri indireceğim" denebilir.
- **Derinlik:** Tohum URL'lerin bulunduğu seviye bir ağacın kökü (root) gibi düşünülürse, "kökten başlayarak 6 seviye ilerleyeceğiz" denebilir. 6 seviye ilerledikten sonra tarayıcı durur.

Hoca süreye dayalı koşulun sorunlu olabileceğini özellikle belirtti. Çünkü tarayıcının çalışma süresinin içinde ağ gecikmesi (network latency) de vardır. Ağ gecikmesi, HTTP isteğinin sunucuya ulaşması ve sunucudan gelen HTTP yanıtının (HTTP response) istemciye ulaşması için geçen süredir. Bu süre, sunucunun web ortamında bulunduğu noktaya göre değişir. Dolayısıyla süre dışında başka durdurma koşulları da seçmek gerekir.

> **Ek bilgi (derste anlatılmadı):** Hocanın kullandığı ağaç benzetmesi, tohumdan başlayan taramayı seviye seviye anlatmak içindir. Web'in kendisi ağaç değil, döngüler içerebilen yönlü bir çizgedir. Bir sayfaya birden çok yoldan ulaşılabilir. Aynı sayfanın tekrar indirilmesi sorununun (Bölüm 11.1) kaynağı da budur.

4. Tohum URL Seçimi ve Taramanın Ölçeği

Hoca slayttaki örnek üzerinden bir sonuca vardı. Arama ortamında bir sayfayla ilgili sonuçlar gelmişti. Bu sonucu aramada gösterebilmek için web'i en az üç seviye taramak gerekmişti. Yani yalnızca tek bir sayfayı arama sonucunda gösterebilmek için, tohum URL'lerden başlayarak çok sayıda sayfayı tarayıp saklamak gerekmişti.

Bunun anlamı şudur: Arama sonuçlarının başarılı olmasını istiyorsak olabildiğince çok sayfayı dizinlememiz (indexing) gerekir. Daha doğrusu olabildiğince çok **ilgili (relevant)** sayfayı dizinlememiz gerekir.

> **Hocanın vurgusu:** İlk başta seçilen tohum URL'ler çok önemlidir. Eğlence (entertainment) sayfalarına ulaşmak istiyorsak, bizi bu alandaki en iyi sayfalara götürecek en doğru URL kümesini seçmemiz gerekir. Yanlış tohumla başlarsak web'in başka taraflarını tarar, ulaşmak istediğimiz sayfalara çok daha uzun sürede ulaşırız. Tohum URL sayısı ve web'in ne kadarının taranacağı bilgisi oldukça önemlidir.

Hoca ardından tarayıcıların web üzerindeki ağırlığına değindi. Tarayıcılar, günlük olarak kullandığımız internet bant genişliği (bandwidth) üzerinde sürekli çalışan uygulamalardır. Bing, Google, Yandex ve Yahoo'nun tarayıcıları web'deki farklı sayfalara sürekli HTTP isteği gönderir ve karşılığında HTTP yanıtı alır. Hocaya göre bant genişliğini en yüksek oranda kullanan uygulamalar bunlardır.

> **Ders sonrası not:** Hoca tarayıcıların internet bant genişliğini en yüksek oranda kullanan uygulamalar olduğunu, ilerleyen kısımda da evrensel tarayıcıların bant genişliğinin yüzde 50'sinden fazlasını tükettiğini söyledi. Altyazıda bu iddia için bir kaynak verilmiyor. İnternet trafiğinin önemli bölümünü büyük olasılıkla video akışı gibi uygulamalar oluşturur; bu oranı kesin bir ölçüm olarak almamak gerekir.

5. robots.txt Dosyası

Tarayıcı bir siteye geldiğinde ne olur? Diyelim ki bir sitedeki HTML dosyası tarayıcının taranacak URL listesine düştü ve tarayıcı bu sayfa için istek gönderecek. Hocanın anlatımına göre her web sitesinin sunucusunda `robots.txt` adlı bir dosya bulunur. Bu dosya, o sitede hangi sayfaların taranabileceğini, hangilerinin taranamayacağını belirtir. Tarayıcı bu dosyayı okuduktan sonra sitede yalnız izin verilen sayfaları taramalıdır.

Tarayıcı bu kurala uymaz ve izin verilmeyen sayfalara HTTP isteği gönderirse, sitenin güvenlik duvarı (firewall) uygulaması bu tarayıcıyı işaretler (flag). Hocaya göre bundan sonra tarayıcı bu siteden hiçbir sayfayı dizinleyemez.

> **Ders sonrası not:** Hoca her sitede `robots.txt` olduğunu, tarayıcının yalnız bu dosyada belirtilen sayfaları tarayabileceğini ve kurala uymayan tarayıcının güvenlik duvarınca işaretlendiğini söyledi. Doğrusu büyük olasılıkla şöyledir: Dosya isteğe bağlıdır ve çoğunlukla taranması istenmeyen yolları (`Disallow`) listeler; listelenmeyen yollar taranabilir kabul edilir. Dosyaya uymak da tarayıcının gönüllü davranışıdır. Güvenlik duvarının engellemesi her sitede otomatik gerçekleşen bir sonuç değil, sitenin ayrıca alabileceği bir önlemdir.

6. Tarayıcılar Nerede Kullanılır?

Hoca "Bu uygulamaları neden yazıyoruz?" sorusuyla farklı kullanım alanlarını saydı.

- **Genel arama motorları:** Arama motorlarının çalışabilmesi için web'deki verinin indirilip dizinlenmesi gerekir.
- **Dikey (vertical) arama motorları:** Yemek tarifi, haber (Google News gibi) veya insanların yazdığı yorumlar üzerine odaklanmış arama motorlarıdır. Bunların üzerinde arama yaptığı ters dizinler de yine tarayıcıyla oluşturulur.
- **İş zekâsı (business intelligence):** Bir alandaki rakip firmaların ne yaptığını izlemek için kullanılır.
- **Web sitesi izleme (monitoring):** Belirli sitelerdeki bilgi sürekli takip edilir.
- **E-posta adresi toplama:** Web'de e-posta verisi paylaşan sayfalardan adres toplanır.

Hocanın iş zekâsı örneği bir gıda firmasıydı. Sırf bu iş için kurulmuş şirketler vardır. Bu şirketlere "Şu alanda ürün çıkaracağım, bu alanda ürün çıkaran firmaların güncel olarak ne yaptığını anlık takip etmek istiyorum" dersiniz. Diyelim ki Ülker yeni bir ürün çıkarıyor. Türkiye'de ve dünyada rakip firmaların bu ürünle ilişkili olabilecek ürünleri hakkında günlük rapor almak istiyor. Bunun için bir tarayıcı çalışır. Rakip firmaların sayfaları ve haber portallarında çıkan haberler sürekli taranır. Bu ürüne rakip olabilecek ürünlerle ilgili sayfalar toplanır ve rapor hâline getirilir.

İzleme örneği ise karma (hibrit) arama motorlarıydı. Bu tür sitelerin arka planında birden fazla elektronik e-ticaret sitesi vardır. Hoca bunlara Media Markt, Bimeks ve Teknosa gibi örnekler verdi. Bu sayfalar sürekli izlenir. Belirli tip ürünler için fiyat, özellik, kampanya ve kaç günde kargoya verileceği bilgisi alınır ve tek bir arama motorunda toplanır. Kullanıcı da filtreleme yaparak bir ürünün farklı firmalarda nasıl satıldığını izleyebilir.

E-posta adresi toplama, kötü niyetli kullanımın örneğidir. İstenmeyen e-posta (spam) ve oltalama (phishing) amacıyla insanların e-posta listesini toplayan tarayıcılar vardır.

Hoca bu kısmı daha önce gösterdiği genel bir şekli hatırlatarak bağladı. Tarayıcı, web'deki dağıtık sunuculardaki HTML sayfalarını belirli bir yöntemle bulur ve merkezde yer alan, yine dağıtık bir veri saklama platformunda saklar. Tarayıcının yaptığı budur. Sonrasında bu veri üzerinde metin ve bağlantı analizi (text and link analysis) çalışmaları yapılır. Bir ters dizin tablosu oluşturulur ve her URL için bir sıralama (ranking) değeri hesaplanır. Kullanıcı da artık gerçek zamanlı aramayı bu tablo üzerinde yapar.

7. Tarayıcı Sınıflandırması

Mevcut bütün tarayıcıları tek bir resimde kategorilere ayırmak istersek şöyle bir sınıflandırma (taxonomy) ortaya çıkar:

```
Tarayıcılar
├── Universal (evrensel; Googlebot, MSNbot)
└── Preferential (tercihli; belirli bir amaca yönelik)
    ├── Focused (odaklı; çevrim dışı çalışır, ters dizin kurar)
    └── Topical (konusal; arama anında çalışır)
        ├── Static (önceden belirli dolaşma politikası)
        └── Adaptive (dolaşırken öğrenir)
```

7.1 Evrensel ve Odaklı Tarayıcılar

Evrensel tarayıcılar (universal crawlers), evrensel arama motorlarını destekler. Googlebot ve MSNbot bunun örnekleridir. Tercihli tarayıcılar (preferential crawlers) ise belirli bir amaca yönelik çalışır ve iki temel kategoriye ayrılır: odaklı (focused) ve konusal (topical).

Odaklı tarayıcıyı hoca arama motoru örnekleriyle anlattı. Diyelim ki gittiğiniz arama motoru yalnız elektronik e-ticaret sayfalarındaki ürünleri aramak için kullanılıyor. Hoca ResearchGate'i de örnek olarak düşündü, ama onun yalnız kendi ortamında arama yaptığını söyleyerek çok iyi bir örnek olmadığını belirtti. Asıl kastedilen, farklı sitelerde aynı konuda yayınlanmış HTML sayfalarında arama yapmayı sağlayan motorlardır. Konu yemek tarifi olabilir. Çevrim içi alışveriş olabilir; bu durumda tarayıcı yalnız elektronik değil, bütün e-ticaret siteleri üzerinde çalışır.

Odaklı tarayıcının yaptığı iş, evrensel tarayıcınınkiyle aynıdır: Sayfaları indirir ve bir ters dizin oluşturulmasına olanak verir. Arama da bu ters dizin üzerinde yapılır. Demek ki odaklı ve evrensel tarayıcılar **çevrim dışı (offline)** çalışır. Arama anında çalışmazlar, yalnız aramanın yapılabilmesi için dizin tablosunu oluştururlar.

7.2 Konusal Tarayıcılar

Konusal tarayıcı (topical crawler) ise **gerçek zamanlı**, yani arama anında çalışır. Hocanın örneği şuydu: Kendinizle ilgili, sorguyu girdikten sonra başlayacak şekilde, web'de kimlerin neler konuştuğunu ve isminizin hangi sayfalarda geçtiğini aramak istiyorsunuz. Aramayı girdiğiniz anda tarayıcı çalışmaya başlar.

Tarayıcı sonuç buldukça bunlar sonuç ekranında görünmeye başlar. Sonuçlar sürekli geldiği için bu tür arama ortamlarının kullanıcı arayüzü genelde iki biçimde düşünülür:

- **Akan sonuçlar:** Arayüzde sonuçlar akar. Ürünün sayfasına belli aralıklarla gittiğinizde sonuçların sürekli değiştiğini görürsünüz.
- **Bildirim (notification) tabanlı:** Aramayı girer, e-posta adresinizi de verip başlatırsınız. Sistem belli aralıklarla bulduğu sonuçları e-postanıza gönderir.

7.3 Statik Konusal Tarayıcılar

Konusal tarayıcılar da kendi içinde ikiye ayrılır. Statik olanlar web'i önceden belirlenmiş bir dolaşma algoritmasına göre dolaşır. Bunlar best-first, derinlik öncelikli arama (depth-first search), genişlik öncelikli arama (breadth-first search) veya PageRank olabilir.

Statik ne demektir? Diyelim ki 10 URL elde ettiniz. Bu URL'lerin işaret ettiği HTML sayfalarını belirli bir sırayla indirmeniz gerekiyor. Bunun için bir politikanız olmalıdır. "Ağacı seviye seviye gezeceğim" derseniz genişlik öncelikli aramayı tercih edersiniz. "Bir dal yakaladığımda en dibine kadar gideceğim, sonra yukarı çıkıp başka bir dalı seçeceğim" derseniz derinlik öncelikli arama yaklaşımını seçmiş olursunuz.

7.4 Uyarlanabilir Konusal Tarayıcılar

Uyarlanabilir (adaptive) konusal tarayıcılar, sayfaları dolaşırken elde ettikleri bilgilerden yola çıkarak hareket eder. Hoca işleyişi adım adım anlattı:

1. Bir sayfaya istek gönderilir, sayfa gelir, içinden URL'ler çıkarılır.
2. Aynı sayfadan anahtar kelimeler (keywords) de çıkarılır.
3. Tarayıcıyı başlatırken kullanılan anahtar kelimeler, sayfadan elde edilen kelimelerle karşılaştırılır.
4. Benzerlik oranı yüksekse bu sayfayı indirmek başarılı bir iş olmuştur; doğru sayfa zinciri takip ediliyordur.
5. Sayfadan öğrenilen yeni kelimeler, kullanıcının girdiği kelimenin yanına eklenir.
6. Tarayıcı bir sonraki URL'yi seçerken artık bu zenginleştirilmiş kelime grubunu kullanır.

Hoca bu yaklaşıma pekiştirmeli öğrenme (reinforcement learning) dedi: Tarayıcı, taradığı sayfalardan elde ettiği bilgiyi sürekli kullanır. Evrimsel (evolutionary) tarayıcılar da bu ailenin başka bir örneğidir.

> **Ders sonrası not:** Hoca, yeni kelimeleri sorguya ekleme işlemine pekiştirmeli öğrenme dedi. Anlatılan mekanizma büyük olasılıkla daha çok sorgu genişletmeye (query expansion) ve ilgililik geri bildirimine (relevance feedback) benzer. Pekiştirmeli öğrenmede genellikle bir ödül sinyaline göre öğrenilen bir eylem seçme politikası vardır.

8. Temel Sıralı Tarayıcı

Tarayıcıların ne olduğunu, türlerini ve motivasyonlarını gördükten sonra hoca basit bir tarayıcıdan başlayarak uygulama konularına geçti. Slayttaki şema, temel sıralı tarayıcıdır (basic sequential crawler). Burada paralelleştirme yoktur. Gösterilen mantıksal akış (logical flow) tek bir iş parçacığı (thread) üzerinde çalışır. Şemadaki her dikdörtgen tarayıcının bir modülüdür.

```
Tohum URL'ler
  -> 1. Sınır listesini başlat
  -> 2. Sınır listesinden URL al
  -> 3. HTTP isteği gönder, sayfayı al (en çok zaman alan modül)
  -> 4. Sayfadan URL'leri çıkar, sınır listesine ekle
  -> 5. Sayfayı saklama ortamına kaydet
  -> 6. Durdurma koşulu sağlandı mı? Hayır ise 2'ye dön
```

8.1 Girdi ve Sınır Listesi

Tarayıcıya en başta bir girdi vermek zorundasınız: "Şu URL'lerden başlayarak web'i tara." Birinci modül, sınır listesini başlatan (initialize frontier) modüldür. Sınır listesi (frontier) genelde

bağlı liste (linked list) olarak uygulanır. Bu modül tohum URL'leri alır ve her biri listenin bir düğümü (node) olacak şekilde bağlı listeyi bellekte (in-memory) kurar.

İkinci modül, sınır listesinden URL çeken modüldür. Listeye gider, en baştaki düğümü alır, bir sonraki modüle vermek üzere URL'yi bu düğümden çıkarır.

8.2 Sayfayı İndirme

Üçüncü modül, elde edilen URL için HTTP isteği oluşturur. HTTP istek nesnesini oluşturur, ilgili uygulama sunucusuna gönderir ve sunucudan HTTP yanıt nesnesini alır. Bu modülle birlikte elimizde URL'ye karşılık gelen HTML dosyası olur.

En çok zaman harcayan modül budur. Çünkü çalışması sırasında ağ gecikmesini de aşması gerekir. Ağ gecikmesi hesaba katıldığında bu modül en fazla zaman alan modüle dönüşebilir.

8.3 URL Çıkarma ve Listeye Ekleme Sırası

Dördüncü modül, gelen HTML belgesinin içinden URL'leri çıkarır ve sınır listesine ekler. Sınır listesi bağlı liste olduğu için yeni URL'ler bu listenin uygun yerlerine eklenmelidir. Uygun yer, listenin hangi politikayla kurulduğuna bağlıdır.

Çıkarma sırası şöyledir: HTML içinde 10 URL varsa ayrıştırıcı (parser) önce ilk gördüğü URL'yi, sonra ikinciyi, sonra üçüncüyü çıkarır. URL'ler bu görülme sırasıyla listeye eklenir. Ama farklı sıralama yöntemleri de olabilir. İleride görüleceği gibi URL'ler gelme sırasından bağımsız olarak popülerliklerine dayalı biçimde de listeye eklenebilir.

> **Ders sonrası not:** Hoca burada "son giren ilk çıkar (LIFO) bir liste kurduysam yeni URL'leri listenin sonuna eklerim" dedi. Dersin bu kısmında URL'ler listenin başından çekiliyor; sona ekleyip baştan çekmek ise ilk giren ilk çıkar (FIFO) davranışıdır. Hoca birkaç dakika sonra genişlik öncelikli aramada bunu doğru biçimde FIFO olarak anlatıyor.

8.4 Sayfayı Saklama

Beşinci modül, indirilen HTML sayfasını dağıtık bir saklama ortamında saklar. Amaç, tarama bittikten sonra bu sayfalar üzerinden bir ters dizin oluşturabilmektir. Bunun için elde edilen bütün sayfaların saklanması gerekir.

Ancak HTML sayfalarında bizim için gerekli olmayan birçok şey vardır: resimler, çoklu ortam (multimedia) nesneleri ve metinle ilgisi olmayan başka içerikler. Bu yüzden sayfalar da belirli bir yönteme göre saklanır. İki seçenek anıldı:

- Sayfayı ifade eden anahtar kelimeler çıkarılır, bir anahtar kelime vektörüne dönüştürülüp saklanır.
- Sayfanın içinden metin çekilir, yalnız metin dosyası olarak saklanır.

Demek ki tarama veri kümesi (crawl dataset) farklı biçimlerde oluşturulabilir.

8.5 Durdurma Koşulu

Son adımda tarayıcı durdurma koşulunu kontrol eder. Koşul, taranacak toplam sayfa sayısı, indirilen sayfaların bayt olarak toplam büyüklüğü veya tohumlardan itibaren kaç seviye aşağı inileceği olabilir. Koşul sağlanmamışsa tarayıcı, sınır listesinden yeni URL alan modülün olduğu yere döner ve işlemi sürdürür.

9. Statik Dolaşma Stratejileri: BFS ve DFS

9.1 Genişlik Öncelikli Arama

Yaygın kullanılan statik yaklaşımlardan biri genişlik öncelikli aramadır (breadth-first search, BFS). Bu tarayıcıda sınır listesi üzerinde ilk giren ilk çıkar (first in first out, FIFO) tabanlı bir kuyruk (queue) uygulanır. Listeye ilk giren URL, listeden ilk çekilen URL olur.

Hoca işleyişi slayttaki örnekle anlattı:

1. Birinci URL'nin sayfasına gidilir. Bu sayfadan iki URL çıkar.
2. Bu iki URL, elimizdeki listenin en sonuna eklenir.
3. Elde edilen URL'lerin her biri için sayfalara gidilir. İlk gidilen sayfanın URL'leri listeye eklenir.
4. Sonra ikinci sayfadan çıkarılan URL'ler yine listeye eklenir.

Her yeni sayfadan çıkan URL'ler listenin sonuna eklenir ve tarayıcı listenin başındaki URL'yi çekerek çalışır. Sonuç "ilk giren ilk çıkar" stratejisidir. Bu strateji web'i seviye seviye taramamızı sağlar.

9.2 Derinlik Öncelikli Arama

Diğer yöntem derinlik öncelikli aramadır (depth-first search, DFS). Tohum URL'den başlanır, bir dal seçilir ve o dalın sonuna kadar ilerlenir. En dipteki yapraklar indirildikten sonra bir sonraki dala geçilir ve o dalın yapraklarına ulaşılır. Bu, son giren ilk çıkar (last in first out, LIFO) yaklaşımıdır.

Uygulamada şöyle ilerler:

1. Birinci URL'ye HTTP isteği gönderilir.
2. Gelen sayfadan çıkan ilk URL için yeni bir istek gönderilir.
3. O sayfadan çıkan ilk URL için yine istek gönderilir. Bu, durdurma koşulundaki derinliğe kadar sürer.
4. Bu noktadaki sayfa indirildikten sonra bir adım geri dönülür. Aynı sayfadaki diğer URL çekilir.
5. Bu seviye bitince bir adım daha geri dönülür. O seviyedeki HTML sayfasında daha önce bakılmamış URL'nin sayfasına gidilir.

Hoca, belirli bir konudaki sayfaları en dip noktasına kadar taramak istediğimizde bu yaklaşımın kullanılabileceğini söyledi.

BFS ve DFS statik çizge dolaşma (graph traversal) algoritmalarıdır. Yazdığınız tarayıcının yapılandırma dosyasında bunlardan biri seçilebilir.

10. Örnek Bir Tarayıcı Uygulaması

Hoca slaytta Perl ile yazılmış temel bir tarayıcı uygulamasını satır satır anlattı. Akış şöyledir:

1. Tohum URL'ler bir dosyaya konmuştur. Dosyadan okunur ve bağlı liste olarak düşünülen sınır listesine atılır.
2. Sınır listesi üzerinde bir döngü kurulur. Durdurma koşulu döngüdedir: Taranacak toplam bağlantı sayısı en büyük değeri geçmemelidir.
3. Döngü içinde listenin en başındaki URL çekilir ve "indirilecek bir sonraki bağlantı" değişkenine atanır.
4. Bu URL'ye HTTP isteği gönderilir. Gelen yanıt `page` adlı bir değişkene atanır.
5. Bu sayfa dizinlenir, yani saklama ortamına konur.
6. Sayfanın içinden bağlantılar çıkarılır. Sayfanın kendi bağlantısı bu sırada dışarıda bırakılır.
7. Bağlantılar `links` değişkenine atanır.
8. `links` işlenir, URL'ler düzgün biçime (canonical format) çevrilir ve sınır listesine eklenir.
9. Bu döngü durdurma koşuluna ulaşılana kadar sürer.

Altıncı adımın gerekçesi şudur: Bir sayfa kendi kendisine işaret edebilir, yani içinde kendi URL'si geçebilir. Bu URL'ye zaten gidip sayfayı getirdik. Bu yüzden yalnız diğer URL'ler alınır.

Sekizinci adım için hoca, URL'nin kök hâlinin bulunması gerektiğini söyledi: Soru işaretinden sonraki anlamsız kısımlar veya URL'nin içindeki sorgu ifadeleri çıkarılır, yalnız kök URL tespit edilip listeye eklenir. Düzgünleştirme konusuna az sonra dönüldü.

> **Ders sonrası not:** Hoca, düzgünleştirmede soru işaretinden sonraki sorgu ifadelerinin çıkarıldığını söyledi. Bu her zaman güvenli değildir. Dinamik sayfalarda farklı sorgu değerleri büyük olasılıkla farklı içeriklere işaret eder; sorgu kısmını körlemesine silmek bu sayfaları tek URL'ye indirebilir.

11. Uygulama Sorunları

Tarayıcı bu işi yaparken dikkat edilmesi gereken bir dizi uygulama ayrıntısı vardır.

11.1 Aynı Sayfayı İki Kez İndirmemek

Bir sayfayı iki kez getirmek (fetch) istemeyiz. Bu yüzden sınır listesine eklenen URL'leri takip etmemiz gerekir. Ancak URL'ler listeye sürekli eklenirken listenin en tepesindeki URL de sürekli çekilir. Dolayısıyla daha önce listede olmuş ama şu anda listede olmayan URL'ler vardır. Yani yalnız listeye bakmak yetmez; bu URL'ler daha önce indirilmiş olabilir.

Çözüm bir **karma tablosu (hash table)** tutmaktır. Bu tabloda daha önce indirilen bütün sayfaların bilgisi yer alır. Tablo bir sayfanın gezildiğini söylüyorsa bu sayfa tekrar sınır listesine eklenmez.

11.2 Sınır Listesinin Hızlı Büyümesi

Sınır listesi çok hızlı büyüyebilir, çünkü her indirilen HTML sayfasından sürekli yeni URL'ler çıkar ve listeye eklenir. Listedeki her URL'yi indirmek zaman aldığı için liste büyüdükçe bu süre de uzar. Oysa tarayıcının belirli bir sürede en önemli sayfaları indirmesini isteriz. Bu yüzden listedeki URL'ler belirli bir mantığa göre sıralanmalıdır.

BFS ve DFS statik tarayıcılardı: Web ağacını nasıl taramak istiyorsak URL'leri geliş sırasına göre o şekilde listeye koyuyorduk. Öncelikle en yüksek popülerlikteki sayfaları indirmek istiyorsak liste sıralamasını popülerlik bilgisine dayandırmamız gerekir.

11.3 İndiricinin Sağlamlığı

İndirici (fetcher) modülünün işini belirli bir zamanda yapması garanti altına alınmalıdır. Bant genişliği çok düşük bir sunucudaki sayfaya ulaşmak istediğimizi düşünelim. Bu sunucudaki sayfayı indirmek için gereken ortalama süre çok büyür. Oysa indiricinin sağlam (robust) olmasını isteriz: Bir sayfa için çok fazla zaman harcamamalı ve başarısız duruma düşmemelidir.

Bunun için indiriciye kurallar konur:

- **Zaman aşımı (timeout):** İndirici bir URL'yi belirli bir sürede indiremiyorsa vazgeçer ve bir sonraki URL'ye geçer.
- **Boyut sınırı:** Sayfa belirli bir büyüklüğü aşıyorsa indirilmeyebilir. Hoca arasından sonra başka bir seçenek daha andı: Çok büyük sayfanın yalnız belirli bir bölümü indirilir.

11.4 Dizinlenebilir Dosyalar

İndirdiğimiz sayfaların dizinlenebilir olmasını isteriz. URL'ler yalnız HTML sayfalarına işaret etmez; PDF sayfalarına, ikili (binary) dosyalara, Word dosyalarına ve metin dosyalarına da işaret eder. Bizim peşinden gitmemiz gereken dosyalar, ayrıştırılıp içinden anahtar kelime çıkarılabilen dosyalardır. Bir URL istemediğimiz tipte bir dosyaya işaret ediyorsa bu dosyayı indirmekten vazgeçmemiz gerekir.

11.5 Ayrıştırma

Kullandığımız web tarayıcıları (browser) akıllıdır. Bir HTML dosyasında eksik kısımlar olsa bile kullanıcıya doğru biçimde gösterebilirler. Eksikleri tespit edip kendileri doldururlar veya eksiklere rağmen sayfayı olması gerektiği gibi görüntülerler.

Tarayıcımız (crawler) da aynı mantıkla çalışmalıdır. Sonuçta HTML dosyasının içinden URL'leri ve metni çıkarıyoruz. Kodu yazarken dosyanın eksik ve sorunlu olabileceğini düşünmemiz gerekir.

11.6 Durak Kelimeler

Bu konu daha önce içerik madenciliğinde (content mining) görülmüştü. Sayfalarda kendi başına anlamı olmayan, yalnız dilin gramer kuralları gereği kullanılan kelimeler vardır. Bunlar durak kelimelerdir (stop words). Saklama ortamına kaydederken bu kelimeler çıkarılabilir.

11.7 Kök Bulma

Tarayıcı çalıştıktan sonra bütün sayfalar saklama ortamına konur ve dizin bu sayfalar üzerinden oluşturulur. Dizinle ilgili uygulama ayrıntısı da daha önce konuşulmuştu: Kelimeler köklerine indirilir ve dizine kök hâliyle yerleştirilir. Böylece aynı kökten türemiş çok sayıda kelime için dizinde ayrı yer tutmak gerekmez ve dizinin boyutu küçük kalır.

> **Ders sonrası not:** Hoca, birbirine eş ve benzer anlamlı kelimelerin tek köke dönüştürüldüğünü de söyledi. Kök bulma (stemming) büyük olasılıkla aynı kökten türemiş kelimeleri birleştirir; kökleri farklı eş anlamlı kelimeleri tek biçime indirmez. Eş anlamlıların birleştirilmesi ayrı bir işlemdir.

11.8 Statik ve Dinamik URL, Örümcek Tuzakları

Web'i tararken karşılaşılan URL'lerin statik mi dinamik mi olduğunu tespit etmemiz gerekir. Dinamik URL'ler için kurallar koymalıyız, çünkü bir kısmı **örümcek tuzağı (spider trap)** oluşturur. Bunların içinde CGI betikleri veya başka betikler olabilir.

Tuzağı hoca şöyle anlattı: Bir web sitesine giriyorsunuz. Bir dinamik URL'ye basıyorsunuz, başka bir sayfa açılıyor. Onun içinde başka dinamik URL'ler var. Birine daha basıyorsunuz, yine başka bir sayfaya gidiyorsunuz. Site bu şekilde tarayıcı uygulamasını kendi içinde sürekli dolaşmaya zorlar.

Hoca iki önlem verdi:

- **URL uzunluğu sınırı:** Dinamik URL'nin uzunluğu belirli bir karakter sayısını aşıyorsa sınır listesine eklenmez. Çünkü böyle bir URL spam URL'si olabilir. Tuzakta oluşturulan URL'lerin karakter sayısı da belli bir sınırı aştığında bu kural devreye girer.
- **Aynı siteden taranan URL sayısı sınırı:** Tarayıcı bir sitede çok yüksek sayıda URL taramaya başladıysa, belli ki site sürekli yeni dinamik URL yaratıp tarayıcıyı aynı alan adı (domain) içinde tutuyor. Bu durumda aynı siteden taranmış URL sayısına bakılır ve sitedeki diğer dinamik URL'ler sınır listesine alınmayabilir.

11.9 Göreli URL'yi Mutlak URL'ye Çevirmek

Bir URL sınır listesine konmadan önce mutlak URL'ye (absolute URL) dönüştürülmelidir. Sayfanın içindeki bağlantılar tam URL olmayabilir; bağlantıda yalnız bir dosya adı, örneğin altyazıda "intle.html" diye geçen bir sayfa adı bulunabilir. Web tarayıcısı (browser) bu bağlantıyla gidebilir, çünkü hedef aynı alan adı ve aynı kategori altındadır.

Bizim yapmamız gereken, göreli URL'yi (relative URL) temel URL (base URL) ile birleştirip mutlak URL'ye çevirmek ve ondan sonra sınır listesine eklemektir.

11.10 URL Düzgünleştirme

Son konu düzgünleştirmedir (canonicalization): URL standart biçime dönüştürülür. Hoca slaytta dört farklı URL gösterdi ve bunların aslında aynı olduğunu söyledi. Farklar şunlardı:

- Bir URL küçük harflerle, bir başkası büyük harflerle yazılmıştır.
- Birinde port özellikle belirtilmiştir. Oysa bu standart porttur ve belirtmeye gerek yoktur.
- Birinde önce bir alt dizine girilip sonra köke dönülüyormuş gibi bir yol parçası vardır. Bu kısım da atılabilir.

Dört URL'nin de işaret ettiği standart biçim (canonical form) tektir. Tarayıcı bunlarla karşılaştığında aynı URL olduklarını tespit edecek şekilde kodlanmalı ve listeye de buna göre eklemelidir. Slaytta başka düzgünleştirme örnekleri de vardı: Her ikili için yukarıdaki değil, aşağıdaki biçim standart olarak seçiliyordu. Bunların hepsi uygulamada birer kural olmalıdır.

> **Ek bilgi (derste anlatılmadı):** HTTP için standart port büyük olasılıkla 80'dir, HTTPS için 443'tür. URL standardında şema ve ana makine adı büyük-küçük harfe duyarsızdır. `..` gibi yol parçaları ise yol normalleştirmesiyle kaldırılır.
>
> **Ders sonrası not:** Hoca küçük ve büyük harfle yazılan URL'lerin aynı olduğunu genel bir kural gibi anlattı. Bu büyük olasılıkla yalnız şema ve ana makine adı için

güvenlidir. Yol kısmı sunucuya göre büyük-küçük harfe duyarlı olabilir, bu yüzden yol kısmının harfleri körlemesine değiştirilmemelidir.

12. Sayfa Deposu

Saklanacak HTML belgesi sayısı çok fazlaysa dosyalar sıkıştırılarak veya her sayfayı temsil edecek nitelikte bilgi tutularak saklanır. Yani sayfanın tamamı yerine yalnız önemli metinler tutulur. Sayfa deposu (page repository) için seçenekler şunlardır:

- **Dosya sistemi, sayfa başına bir dosya:** Her URL bir karma fonksiyonuyla (hash function) dosya adına dönüştürülür. Her sayfa ayrı bir dosya olur. Sonuçta tarama verisinde çok fazla ayrı dosya oluşur.
- **Tek büyük XML dosyası:** İndirilen her sayfa XML'e dönüştürülür. Dosyanın her elemanı bir sayfayı ifade eder. Bu da dosya sistemi üzerinde duran bir yaklaşımdır.
- **İlişkisel veritabanı (relational database):** Sayfalar standart bir veritabanında tutulur.
- **Bu işe adanmış veritabanları:** Veritabanı yaklaşımını kullanır, ama yükleri (overhead) standart ilişkisel veritabanlarından düşüktür. Hocanın verdiği örnek altyazıda "birthday DB" diye geçiyor; bu büyük olasılıkla Berkeley DB'dir.

13. Eş Zamanlı Tarayıcılar

13.1 Sıralı Tarayıcı Nerede Zaman Harcar?

Şimdiye kadar konuşulan yapı sıralı tarayıcıydı ve paralellik içermiyordu. Oysa hocaya göre günümüzde uygulanmış hiçbir tarayıcı sıralı çalışmaz. Web'de taranması gereken çok fazla belge vardır, bu yüzden çok sayıda URL'yi aynı anda indirip işleyebilen tarayıcılara ihtiyaç vardır. Eş zamanlılığı (concurrency) uygulamak için önce sıralı tarayıcının nerede çok zaman harcadığını incelemek gerekir. Hoca üç nokta saydı:

- **DNS sorgusu:** HTML dosyasından bir URL çektik ve HTTP isteği göndereceğiz. Bunun için URL'deki ana makine adına (hostname) karşılık gelen IP adresini bilmemiz gerekir. Bu eşlemeyi alan adı sunucusu (Domain Name Server, DNS) yapar. Tarayıcı ona "Şu ana makine adına karşılık gelen IP adresini verir misin?" diye istek gönderir. Bu işlem zaman alır.
- **Senkron soket:** İstek standart soket sınıfıyla gönderilmişse işlem senkron gerçekleşir. Yanıt gelene kadar soket kilitlenir (block) ve başka bir iş parçacığı tarafından kullanılamaz.
- **Ağ süresi:** İsteği gönderdikten sonra yanıt gelene kadar geçen süredir.

Hoca bu üçünün de paralelleştirilmesi veya hızlandırılması gerektiğini söyledi.

13.2 Soket Hatırlatması

Hoca ağ protokollerini hatırlattı. Bir uygulamayı başka bir makinedeki uygulamayla TCP/IP üzerinden konuşturmak istediğinizde, bu protokolleri kullanmanızı sağlayan kütüphaneler size bir soket sınıfı verir. Bu sınıftan bir nesne oluşturursunuz. Nesnede karşı tarafın IP adresini ve portunu belirtirsiniz. Göndermek istediğiniz mesajı da bu soket üzerinden gönderirsiniz. Soket senkron ise istek gönderildikten sonra yanıt gelene kadar başkası tarafından kullanılamaz.

13.3 Eş Zamanlı Tarayıcının Yapısı

Slayttaki eş zamanlı tarayıcı (concurrent crawler) şemasında gri arka planlı her dikdörtgen bir iş parçacığıdır. Her iş parçacığı, sıralı tarayıcıdaki mantıksal akışın tamamını içerir.

```
Tohum URL'ler
  -> Sınır listesi yöneticisi (Frontier Manager; paralelleştirilmez)
       tohumları listeye ekler, bütün iş parçacıklarını başlatır

İş parçacığı 1..N (aynı anda çalışır):
  sınır listesinden URL al -> indir -> URL'leri çıkar
  -> depoya kaydet -> durdurma koşulunu kontrol et -> başa dön
```

```
Paylaşılan yapılar: sınır listesi ve sayfa deposu (erişim senkronize)
```

Sınır listesi yöneticisi (frontier manager), ilk başlangıçta yapılması gereken işleri yapan modüldür. Bu yüzden paralelleştirilemez. Tohum URL'leri listeye o ekler ve bütün iş parçacıklarını o çalıştırır.

Her iş parçacığı ayağa kalkınca listeden URL alır, sayfayı getirir, içindeki URL'leri çeker, sayfayı saklar ve durdurma koşuluna bakar. Sonra listeden yeniden URL alır. Bütün iş parçacıkları bunu aynı anda yapar. Böylece listeden eleman alıp işleme adımı çok sayıda eş zamanlı iş parçacığıyla yürütülmüş olur.

Yine de iki yavaş nokta kalır. Sayfayı getirirken DNS sunucusuna URL'nin IP adresini sormak hâlâ zorunludur. Sayfayı depoya yazarken de giriş/çıkış erişimi (IO access) gerçekleştiği için zaman alıcı bir işlem vardır. Bunlara da çözüm bulmak gerekir.

13.4 Paylaşılan Veri ve Senkronizasyon

Özetle eş zamanlı tarayıcılarda çoklu iş parçacığı (multi-threading) kullanılır. Her iş parçacığı bir sıralı tarayıcı gibi çalışır. Farkları, aynı veri yapılarını paylaşmalarıdır: Hepsi aynı sınır listesinden okur ve indirdikleri HTML belgelerini aynı depoya yazar.

Bu yüzden paylaşılan verinin olduğu yerlerin senkronize edilmesi gerekir. Birden fazla iş parçacığı sınır listesine ulaşıyorsa buraya senkronizasyon konur ve aynı veriye aynı anda erişilmesi önlenir. Depoya yazarken de senkronizasyon gerekir.

14. Evrensel Tarayıcılar

14.1 Temel Özellikler

Evrensel tarayıcılar, evrensel arama motorlarını destekler ve çok büyük web verisine ölçeklenebilir. Hocaya göre günümüz ağ bant genişliğinin neredeyse yüzde 50'sinden fazlasını tüketirler (bu iddia için Bölüm 4'teki ders sonrası nota bakın).

Dizini her seferinde sıfırdan oluşturmazlar, çünkü bu çok zaman alır. Mevcut dizine artımlı (incremental) eklemeler yaparlar: Yeni karşılaştıkları sayfaları dizinler ve mevcut dizinleri günceller.

> **Hocanın vurgusu:** Evrensel tarayıcı uygulanırken dikkat edilmesi gereken iki önemli etken vardır: performans ve politika.

Performans açısından tarayıcı milyarlarca sayfayı tarayacak şekilde ölçeklenebilir olmalıdır.

14.2 Politika: Kapsam ve Güncellik

Politika açısından iki farklı istek olabilir: Sayfaların güncel olmasını veya olabildiğince fazla sayfa olmasını talep edebilirsiniz. Yani kapsam (coverage) ile güncellik (freshness) arasında bir denge (trade-off) vardır.

- **Kapsam:** Birbirinden farklı, olabildiğince çok sayfa dizinlenir.
- **Güncellik:** O kadar çok sayfa olmasa da dizinlenen sayfaların içeriği sürekli güncel olur. Tarayıcı dizinlediği aynı sayfayı daha yüksek sıklıkla ziyaret edip içeriği indirir ve günceller. Hiç karşılaşmadığı sayfaları dizinlemek yerine eldeki sayfaların güncelliği üzerinde çalışır.

Evrensel tarayıcı uygularken bu denge için bir politika belirlemek gerekir. Hoca her evrensel tarayıcının bu dengeyi yapılandırılabilir (configurable) hâle getirmesi gerektiğini söyledi. Çünkü kimi zaman kapsam, kimi zaman güncellik önem kazanır. Tarayıcı ikisini de destekleyecek esneklikte olmalıdır.

14.3 Zaman Alan Adımlara Çözümler

Hoca eş zamanlı tarayıcıda kalan yavaş noktaları hatırlattı: DNS sunucusundan IP ile URL eşlemesi, senkron soket kullanımı ve ağ süresi. Evrensel tarayıcılar bunlara çözüm üretmek zorundadır.

Bunların ilki eşzamansız soketlerdir (asynchronous sockets). TCP/IP'nin sunduğu standart senkron soket mantığının yanı sıra eşzamansız soket kullanılır. Hocanın anlatımıyla soket üzerinden karşıya bir istek gönderilir, yanıt beklenmeden aynı soket başka bir istek göndermek için tekrar kullanılır.

> **Ders sonrası not:** Hoca eşzamansız soketi, yanıt beklemeden aynı soketten başka istek göndermek olarak anlattı. Bir TCP soketi tek bir bağlantıya bağlı olduğu için bu anlatım farklı sunucular için geçerli değildir. Asıl kazanç büyük olasılıkla, bir iş parçacığının bloke olmadan çok sayıda açık bağlantıyı aynı anda yürütebilmesidir. Hocanın "makinede açılabilecek soket sayısı artar" sözü de bu yönü işaret eder.

14.4 Evrensel Tarayıcı Mimarisi

Slayttaki mimari, eş zamanlı tarayıcıya benzer. Gri arka planlı her dikdörtgen yine bir tarayıcı iş parçacığıdır. Fark, zaman alan aşamalar için optimizasyon yapılmış olmasıdır:

- **DNS önbelleği (cache):** DNS sunucuları için önbellek mekanizması uygulanır. IP adresiyle URL'yi eşleme daha hızlı olur.
- **Eşzamansız soketler:** Tarayıcının çalıştığı makinede açılabilecek soket sayısı çok artar. Böylece daha fazla sayfa eş zamanlı indirilebilir.
- **Yazılım tabanlı ağ anahtarları (network switches):** Bir sayfaya ulaşmak için gereken yönlendirme (routing) süresi kısalır. Amaç, sayfayı indirme süresini azaltmaktır.
- **Yazılım tabanlı depolama anahtarları (storage switches):** URL'ler çıkarıldıktan sonra sayfa saklama ortamına konurken giriş/çıkış erişimi daha kısa sürede gerçekleşir.

14.5 Dizin Boyutları ve Güncelliğin Ölçülmesi

Evrensel tarayıcıların dizinlerine bakarak web'deki veri miktarı hakkında fikir edinebiliriz. Farklı arama motorlarının tarayıcıları farklı dizin boyutlarına sahip olabilir ve bu boyutlar yıldan yıla değişebilir. Burada tarayıcıların farklı politikaları desteklediği de görülebilir. Arama motorları kimi zaman güncellik politikasını kapsam politikasına tercih edebilir.

İndirilen sayfaların güncelliğini anlamak için sayfaların `Last-Modified` ve `Expires` bilgilerine bakılır. Bunlardan sayfanın ne kadar önce güncellendiği ve bir sonraki yeniden ziyaretin ne zaman yapılması gerektiği belirlenir.

> **Ders sonrası not:** Altyazıdaki "Glass Modified" ifadesi büyük olasılıkla `Last-Modified` bilgisidir. Hoca bunlara sayfa içindeki etiket (tag) dedi; ikisi de büyük olasılıkla öncelikle HTTP yanıtının başlık (header) alanlarıdır.

15. Tercihli Tarayıcılar

15.1 Önem Ölçüsü

Sınır listesi kurulurken FIFO veya LIFO politikaları BFS ve DFS mantığı içindi. Öncelik sayfanın popülerliğine verilmek istenirse, sayfalar listeye PageRank gibi bir popülerlik değerine dayanarak eklenir. Bu değer bir önem ölçüsüdür (importance measure). Elimizde bir sayfanın önemiyle ilgili veri varsa sınır listesindeki sıralamayı buna göre kurabiliriz.

Peki bize bir URL verilip önem değeri sorulursa bu değer nasıl hesaplanır? Hoca üç yol anlattı:

- **Bağlantı tabanlı sıralama:** Elimizde bir web çizgesi varsa veya o ana kadar taranan sayfalardan bir çizge oluşturduysak, bu çizgede bağlantı tabanlı bir sıralama algoritması çalıştırırız. Her sayfanın önem değeri çizge üzerinden hesaplanır.
- **URL metni ile konu benzerliği:** Tarayıcının çalıştırıldığı konuyla ilgili anahtar kelimeler ile URL arasında bir benzerlik fonksiyonu çalıştırılır. URL'nin içinde bulunan kelimeler ile arama konusu kelimeleri karşılaştırılır.
- **Üst sayfa ile konu benzerliği:** URL'nin çıkarıldığı üst (parent) HTML dosyasındaki kelimeler ile arama konusu arasındaki benzerliğe bakılır ve URL'ye buna göre puan verilir.

Demek ki tercihli tarayıcılar URL'lere bir önem puanı verir ve URL'leri bu puana göre sıralar.

15.2 Odaklı Tarayıcı

İki tip tercihli tarayıcı vardır. Odaklı tarayıcı standart tarayıcılar gibi bir ters dizin oluşturur ve arama bu dizin üzerinden yapılır. Konusal tarayıcı ise arama başlatıldığında çalışır ve dizin oluşturmak için çalışmaz.

Gözetimli odaklı tarayıcılar (supervised focused crawlers), karşılaştıkları sayfanın sorumlu oldukları kategoride olup olmadığına bakar. Bunun için bir sınıflandırıcı (classifier) kullanırlar. Hocanın örneği yemek tarifi sayfalarını bulmaya çalışan tarayıcıydı:

1. Elde bir URL vardır. Sayfa indirilir ve içeriği çıkarılır.
2. Sınıflandırıcı sayfayı "yemek tarifidir" veya "değildir" diye etiketler.
3. Yemek tarifi olarak etiketlenirse içindeki URL'ler çıkarılıp sınır listesine eklenir.
4. Değilse bu sayfadan çıkarılan URL'ler eklenmez.

> **Ders sonrası not:** Hoca önce sayfa kategorideyse indirildiğini, değilse indirilmediğini söyledi. Hemen ardından verdiği adımlarda ise sayfa önce indiriliyor, sonra sınıflandırılıyor. Bu örnekte sınıflandırmanın sonucu, sayfadaki bağlantıların listeye girip girmeyeceğini belirler.

15.3 Konusal Tarayıcı

Konusal tarayıcıda, elde edilen URL'nin bağlantı metninin (anchor text) veya URL'nin bulunduğu üst dosyadaki anahtar kelimelerin konuya benzerliğine bakılır. Benzerlik belirli bir derecenin üzerindeyse URL sınır listesine eklenir.

> **Ders sonrası not:** Hoca bağlantı metnini "URL'nin kendi metni" gibi anlattı. Bağlantı metni büyük olasılıkla HTML'deki bağlantının üzerinde görünen metindir; URL dizgesinin kendisiyle aynı şey değildir.

15.4 Algoritmalar

BFS ve DFS aslında tercihli tarayıcı değildir. Standart tarama algoritmalarıdır. Genelde tercihli tarama algoritmalarına karşı kıyaslama (benchmark) algoritması olarak kullanılırlar.

> **Ders sonrası not:** Hoca sınıflandırmayı anlatırken BFS ve DFS'yi tercihli tarayıcıların altındaki statik konusal tarayıcılar arasında saymıştı. Burada ise tercihli olmadıklarını söylüyor. Doğru okuma, ikisinin konudan bağımsız statik dolaşma stratejileri ve kıyaslama ölçütleri olduğudur.

Tercihli algoritmalar şunlardır:

- **Best-N-First:** Altyazıda "best and first" ve "en tane URL" diye geçiyor; adı büyük olasılıkla budur. İlk girilen konuya benzer sayfalardan elde edilen URL'lere daha büyük önem değeri verilir. Sınır listesi benzerlik değerine göre yukarıdan aşağıya sıralı olur. Listedeki ilk N URL aynı anda, paralel olarak indirilmeye çalışılır.
- **PageRank tabanlı:** Sınır listesindeki URL'ler anahtar kelimelere ve PageRank değerlerine göre sıralanır.
- **Shark Search:** Bağlantı metnindeki kelimelere ve URL'nin bulunduğu üst sayfadaki kelimelere bakarak URL'nin önem değerini bulur.
- **Fish Search:** Shark Search'ün önceki uygulamasıdır. Shark Search bunun geliştirilmiş sürümüdür.
- **InfoSpider:** Karşılaştığı yeni sayfalardan çektiği yeni URL'lere dayanarak web'deki sayfaları arar. Konusal tarayıcı algoritması örneğidir.

  Ek bilgi (derste anlatılmadı): InfoSpiders, büyük olasılıkla Menczer ve arkadaşlarının evrimsel bir tarayıcı yaklaşımıdır. Tek bir tarayıcı yerine bir ajan popülasyonu kullanır. Ajanlar ilgili sayfa buldukça ödüllendirilir, başarılı olanlar çoğalır, başarısız olanlar elenir. Bu yüzden hocanın uyarlanabilir tarayıcılar başlığında andığı evrimsel tarayıcılarla da ilişkilidir.

16. Tercihli Tarayıcıları Karşılaştırmak: Tarama Boyutu ve Duyarlılık

Tercihli tarayıcıların çalışmasını karşılaştırırken tarama boyutu-duyarlılık (crawl size-recall) grafikleri kullanılır. Grafikte tarama boyutu sıfırdan yüzde yüze gider. Bir anahtar kelime girildiğini ve bu kelimeye karşılık gelen sayfaların en kısa sürede indirilmek istendiğini düşünelim. Hocanın gösterdiği grafikte dört farklı tarayıcı vardı:

- PageRank'e bakan tarayıcı.
- Genişlik öncelikli arama uygulayan tarayıcı.
- Elindeki URL'leri rastgele (random) seçip indiren tarayıcı.
- Sayfaya işaret eden geri bağlantılara (backlinks) bakan tarayıcı.

Duyarlılık (recall), elde edilen ilgili sayfaların toplam ilgili sayfalara oranıdır:

$$\text{duyarlılık} = \frac{\text{elde edilen ilgili sayfa sayısı}}{\text{toplam ilgili sayfa sayısı}}$$

Yüzde 100'de bütün sayfalar indirilmiştir. O noktada her tarayıcı, en başta belirlenen anahtar kelimeye karşılık gelen bütün belgeleri indirmiş olur. Yani bu nokta algoritmaları ayırt etmez.

> **Hocanın vurgusu:** Asıl hedef şudur: Tarama boyutu daha düşükken, yüzde 20'lerde, yüzde 40'lardayken, hangi algoritma anahtar kelimeye karşılık gelen en fazla sayfayı indiriyor? Düşük tarama boyutunda duyarlılığı yüksek olan algoritma tercih edilmelidir.

Hocanın gösterdiği grafikte PageRank tabanlı algoritma, düşük tarama boyutlarında bile en fazla ilgili sayfayı indiriyordu.

17. Topical Tarayıcı Örneği

Son olarak hoca konusal tarayıcıya döndü. Bu tarayıcıya iki girdi verilir:

- **Konu (topic):** Sorgu biçiminde, bir açıklama veya anahtar kelime olabilir.
- **Tohum sayfa kümesi (set of seed pages):** Tarayıcıya "Bu tohum URL'lerden başlayarak bu konuya dayalı olarak web'de ara" denir.

Tarayıcıdan bir dizin oluşturması istenmez. Arama komutu verildiği anda taramaya başlaması istenir.

Slayttaki örnek, InfoSpider algoritmasını uygulayan bir sistemdi. Anahtar kelimeler girilir, tarayıcı tohum URL'lerden başlayarak aramaya koyulur. Bulduğu sonuçlara girilen kelime grubuna yakınlıklarına göre skor atar ve yukarıdan aşağıya sıralı biçimde gösterir. Bu ekran, akan sonuçların sürekli güncellendiği bir kullanıcı arayüzüdür. Yeni sonuçlar bulundukça sonuç listesinde olması gereken yere yerleşir.

18. Açık Kaynak Tarayıcılar

Slaytta, projelerde kullanılabilecek açık kaynaklı tarayıcı kodlarının bağlantıları vardı. Hoca yaygın olarak kullandığı tarayıcının adını da verdi. Altyazıda "notch" diye geçen bu tarayıcı büyük olasılıkla Apache Nutch'tır. İçinde kendi arama motorunu da getirir. Bunun dışında farklı programlama dillerinde yazılmış tarayıcılar da vardır.

Hoca bir uyarı ekledi: Bu araçlar olduğu gibi (by default) kullanılamaz, üzerlerine kod yazmak gerekir.

Dersin Toparlaması

Bu derste bir tarayıcının neden gerekli olduğundan başlayıp nasıl yazılacağına kadar gidildi. Arama ters dizin üzerinde yapıldığı için sayfaların önce toplanması gerekir. Tarayıcı tohum URL'lerden başlar, sayfaları indirir, yeni URL'ler çıkarır ve durdurma koşuluna kadar devam eder. Sınır listesinin nasıl sıralandığı tarayıcının davranışını belirler: FIFO genişlik öncelikli, LIFO derinlik öncelikli taramayı verir, önem puanına göre sıralama ise tercihli taramayı. Uygulama sorunları tekrar indirmeden örümcek tuzaklarına ve URL düzgünleştirmeye kadar uzanır.

Ölçek büyüdükçe çözüm de değişir. Sıralı tarayıcıdan çok iş parçacıklı eş zamanlı tarayıcıya, oradan DNS önbelleği, eşzamansız soket ve yazılım tabanlı anahtarlar kullanan evrensel tarayıcıya geçilir. Evrensel tarayıcıda kapsam ile güncellik arasında bir politika seçilir. Tercihli tarayıcılar ise önem ölçüsüne göre çalışır ve düşük tarama boyutundaki duyarlılıklarıyla karşılaştırılır. Gelecek derste 7. bölüm olan bağlantı analizine (link analysis) geçilecektir.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.